

מתג (ראוטר) switch

רכיבים קודמים

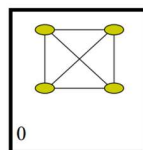
1. בשכבה 1 היה לנו repeater/hub שתפקידו היה להאריך את הכבל. הוא משחזר את האות (אולי גם משכפל אותו) ומחזיר אותו הלאה.
2. בשכבה 2 היה משהו יותר מתוחכם – bridge/ethernet switch – שלא רק חיבר רשתות אלא גם עשה פעולה של סינון. לא כל הודעה הגיעה לכולם אלא רוב ההודעות יגיעו רק לרשת שצריכה לשמוע אותן. לשם כך היו כתובות MAC.
3. בשכבה 3 יש **מתג** – כאן יהיה לנו הרבה עצים פורסים. לא כדי להימנע מלולאות אלא כדי לדעת איך להגיע ליעד, כל יעד יהיה שורש של עץ פורש כזה. המידע יהיה מאוחסן בטבלאות הניתוב.

בשכבת האפליקציה אנחנו כותבים תכנית ויכולים לשלוח הודעה מכל מקום לכל מקום. אפשר לשים את זה ברשת וירטואלית על הרשת הפיזית.

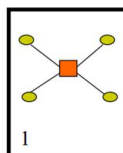
ככל שהשכבה יותר גבוהה, אפשר לספק פונקציונליות יותר עשירה. מצד שני, הביצועים/המחיר של הפונקציות האלה הולך וגדל.

איך אפשר להעביר הודעות?

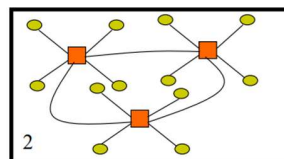
- לחבר כל תחנה לכל תחנה – אבל זה לא scalability. כל תחנה שנוספת צריך לחבר אותה לתחנות אחרות ונקבל overhead ליניארי במס' התחנות.



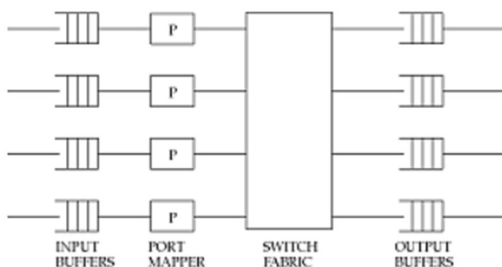
- לקחת את כל הרשת ולכווץ אותה לתוך קופסה (בדומה לhub). הקופסה תהיה מתג בעלת כמה יציאות (ports) והקופסה תדע להעביר כל הודעה שמגיעה מאיזה שהיא רגל ליעד שלה – זו למעשה פעולת המיתוג.



- לקחת ולחבר מתגים בינן לבין עצמם. כך המיתוג מתבצע במס' מקומות.



מבנה מתג



- **באפרים** – על קווי הכניסה יש באפרים שמאחסנים את ההודעות עד שנוכל לטפל בהן. יכולים להיות גם באפרים ביציאה.
 - **Port mapper** – אומר באיזה port הודעה צריכה לצאת כדי להגיע ליעד.
 - **Switch fabric** – יש שם את הרשת הפנימית שהמתג מממש. מתג זה בעצם רשת בתוך קופסה עם כל מני עזרים.
- כל אלמנט הוא אופציונאלי ויכול להיות טריוויאלי. נראה אפשרויות שונות למימוש.

סוגי מיתוג

1. מיתוג מעגלי circuit switching

- המיתוג מתבצע ע"י זה שיוצרים מעגל מהשולח עד המקבל.
- יש רוחב פס קבוע שמקצים בהתחלה ונשאר כך עד סוף הקשר.
- מקצים את המסלול בעת החיבור ולכן לכל פקטה אין header. יודעים מאיפה היא מגיעה וכך יודעים לאן היא תגיע.

2. מיתוג מנות packet switching

- באן אין שיריון של הקווים flow מסוים. מקבלים פקטה ושולחים אותה ליעד שלה.
- אין רוחב פס קבוע.
- באן המתג לא יודע לאן הפקטה הולכת ולכן לכל פקטה יש header שאומרת לאן צריך להגיע.

הפעולה של העברת הדאטא (forwarding) דומה עבור שני סוגי המיתוג.

תפקידי מתג

1. **Forwarding** – להעביר את המידע ליעד.
2. **Backpressure** – יכול לאותת לשולח להאיט את הקצב במידה והיעד לא מספיק לקבל את החבילות.
3. **Routing** – מחשבים את המסלולים של כל חבילה מכל מקום לכל יעד ואז יוצרים את טבלאות הניתוב (routing table). הטבלאות צריכות להכיל בתוכן את כל היעדים.
4. **Congestion** – לפתור את פקקי התנועה. בניגוד לכבישים במציאות, פקטות לא מגיעות לפי הסדר אחת אחרי השניה אלא יכולות להגיע בכל מני זמנים ויש צורך לחשב איזו חבילה נעכב.
5. **Admission control** – הדרך להבטיח איכות שירות. כל מה שנוגע ל accept/reject.

קריטריונים לביצועים

1. **Capacity** – כמה ביטים המתג יכול להעביר בשניה.
2. **Call blocking** – במיתוג מעגלי יכול להתרגל blocking, כלומר כאשר יש חסימה בין הקלט לפלט. נרצה למנוע מצב כזה.
3. **packet loss** – במיתוג מנות לא יכולה להתרחש call blocking אבל חבילות יכולות ללכת לאיבוד ולכן נרצה למזער את כמות אובדן החבילות.

סוגי חסימות

- **חסימה פנימית** – אי אפשר להגיע מה input port ל output port.

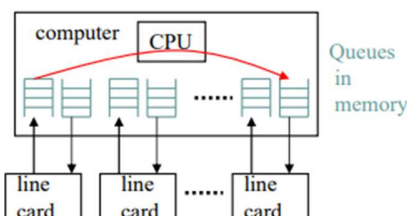
פתרונות:

- עושים הקצאת יתר fabrica – בונים רשת פנימית מהירה יותר או בעלת יותר מסלולים.
- להשתמש בבאפרים. יעיל עבור burst קצרים.
- לגלגל את הבעיה הלאה – כלומר לתת למישהו בהמשך, למשל ל user, והוא יחליט מה לעשות עם זה.
- **חסימה חיצונית** – היציאה בשימוש ואי אפשר להעביר חבילות לשם יותר. לכן החבילות האלו ימתינו בבאפרים. אם אין בבאפר מקום החבילה תלך לאיבוד.

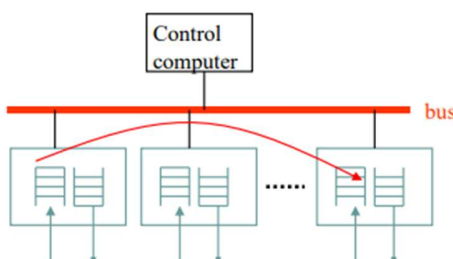
דורות של מתגים

1. דור ראשון

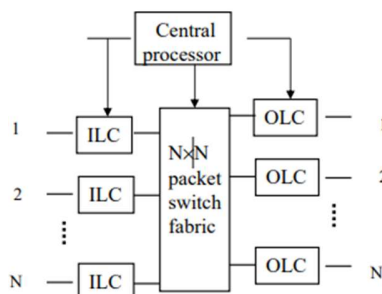
אפשר לקחת מחשב ולהתקין בו מס' כרטיסי רשת. ה-CPU קורא את החבילות הנכנסות, מחליט לאן צריך להוציא אותן לפי טבלאות הניתוב ומעביר אותן. מבחינת כרטיסי רשת לא צריך משהו מתוחכם, ה-CPU הוא זה שעושה את כל העבודה.



2. דור שני – ה-CPU לא מעורב כאן בכלל כרטיסי רשת הוא זה שיפענח את ההודעה, יבין לאן היא צריכה להגיע ויעביר אותה דרך ה-BUS ישירות ליעד (בדומה ל-DMA – direct memory address). רוב המתגים הפנימיים הם דור שני.



3. דור שלישי – יש לו גם כרטיסים שיודעים להסתכל על החבילות וגם יש לו רשת פנימית fabric/interconnect שהוא לא רק BUS אלא משהו יותר מתוחכם. המעבד מקפגג אותו. החבילות הולכות מהכרטיסי רשת באופן ישיר אבל הרשת הפנימית יותר מתוחכמת ויכולה להעביר הרבה מאוד ביטים.

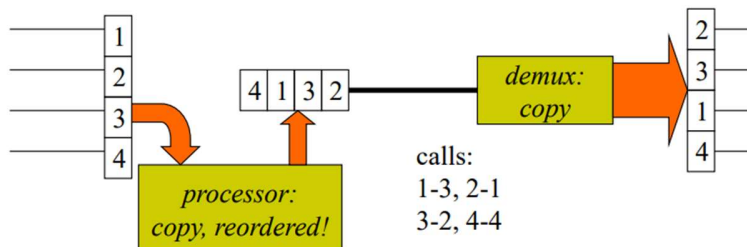


אסטרטגיות מיתוג

1. TSI (Time Slot Interchange)

אפשר לממש מתג בעזרת mux ו-demux.

לוקחים את ארבעת הכניסות של ה-mux ובמקום להוציא אותם באופן שרירותי, נשטח אותם ונוציא אותם לפי הסדר שנקבע ע"י בקשת הניתוב. למשל מה שמגיע בכניסה 1 צריך לצאת ביציאה 3. לאחר מכן זה נכנס ל-demux והוא מפצל אותם בחזרה.



בעיות:

- צריך קו מהיר באמצע. לקחנו n קווים ושיטחנו אותם ולכן צריך קו מהיר יותר פי n.
- אפשר לממש את זה עם זיכרון משותף אבל אז הגישה לזיכרון נהיית צוואר בקבוק משמעותי.

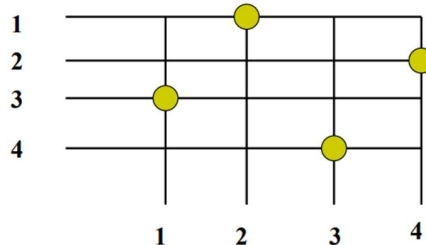
2. Space switching

Space division – מוסיפים קווים נוספים כדי שלהודעות יהיה יותר מסלולים לעבור בהם.

• Crossbar קווים צולבים

קווי הכניסה הם קווים אופקיים וקווי היציאה הם קווים אנכיים. אנחנו מקבלים crossbar וכל נקודת חיתוך היא נקודת מיתוג.

sessions: (1,2) (2,4) (3,1) (4,3)



יתרונות:

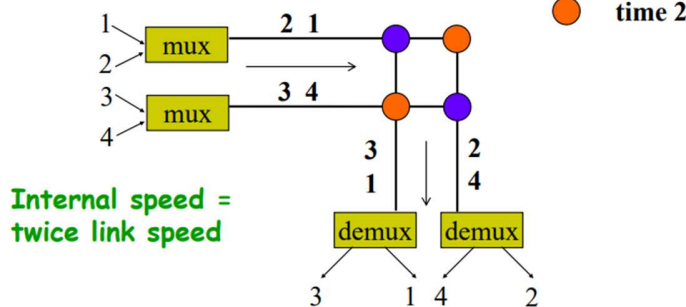
- פשוט למימוש (עושים זאת בVLSI).
- פשוט לבקרה.
- גמיש.

חסרונות:

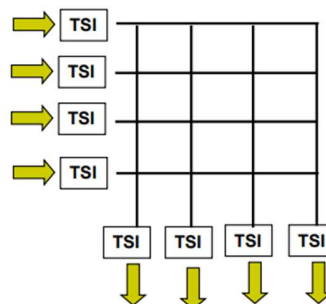
- אי אפשר לעשות את זה גדול מדי כי מס' הנקודות שנרצה למתג הוא ריבועי במס' הכניסות.
- תופס הרבה שטח.
- צוואר בקבוק – איך מקודדים את האינפורמציה של מי הולך לאן. יכול להיות שחוק אחד מספיק לדאטא אבל צריך $\log(n)$ כדי להגיע לו לאן לצאת.

- TS – נשלב את שני הרעיונות – נשתמש בcrossbar ובקווים מהירים. במקום שיהיה סידור סטטי בין קווי היציאה לקווי הכניסה, הcross bar יעבוד בכמה פעימות. זה חוסך גם במהירות הקו וגם בגודל הcrossbar.

Target permutation: $1 \rightarrow 2, 2 \rightarrow 4, 3 \rightarrow 1, 4 \rightarrow 3$



TST (Time Space Time switching) – לפעמים אנחנו רוצים שהסדר ביציאה יהיה כמו הסדר בכניסה. לשם כך מוסיפים עוד TSI.



Schedule

כעת נשכח מהאילוצים הפיזיים ונגדיר את הבעיה כאלגוריתמית.

קלט

- n – מספר הכניסות וגם מספר היציאות.
- σ – פרמוטציה – כל כניסה צריכה להגיע ליציאה אחרת.
- s – הגודל של צד אחד ב crossbar. הגודל הכולל של crossbar הוא $(s \times s)$.
- m – מס' השיחות שצריך לדחוף על כל קו ב crossbar כאשר זה שווה ל $m = \frac{n}{s}$.

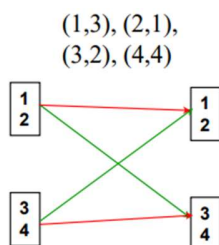
פלט

- Schedule – crossbar עובד בכמה פעימות לכן אנחנו רוצים לחשב schedule באורך t – לוח זמנים שאומר:
 - בכל צעד איזה מהחבילות אנחנו מזרימים בתוך crossbar.
 - על כל קו כניסה מהו סדר הבקשות.

גרף ניתוב

הצמתים – כניסות ויציאות.

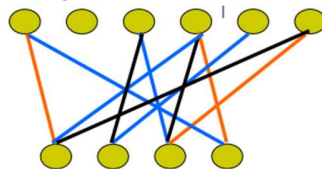
הקשתות – בקשות החיבור (שיחות).



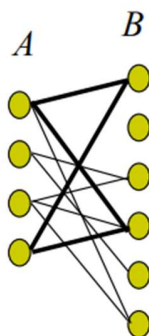
- טענה: למצוא schedule לשיחות זה כמו למצוא צביעת קשתות של גרף הניתוב.
- בפעימה הראשונה נעביר את השיחות שמסומנות באדום ובפעימה השנייה את השיחות שמסומנות בירוק.
- זה לא מפריע כי אין שיחות מאותו צבע עם צומת משותף.

מונחים מתורת הגרפים

- צביעת קשתות – כל זוג קשתות שיש להן קודקוד משותף נבצע בצבע שונה.
- מס' הצבעים חייב להיות לפחות בגודל הדרגה המקסימלית בגרף $\#colors \geq \max degree$.



- גרף דו-צדדי – אין קשתות שמחברות בין צמתים ב-A או ב-B.



- זיווג – קבוצה של קשתות מבודדות. אין קודקוד שמופיע יותר מפעם אחת בקשת.

טענה: משפט הול Hall's Theorem

בגרף דו-צדדי אפשר למצוא לזיווג עבור S אם ורק אם $|N(X)| \geq |X|$ $\forall X \subseteq S$.

במילים אחרות, אם לוקחים תת קבוצה X ומסתכלים על השכנים שלו N (Neighbors), כדי שיהיה זיווג מס' השכנים צריך להיות לפחות בגודל הקבוצה X .

טענה: משפט קניג König's Theorem

בגרף דו-צדדי עם דרגה מירבית Δ אפשר לצבוע את הקשתות שלו ב- Δ צבעים.

במילים אחרות, בגרף דו-צדדי, הגודל של הזיווג המירבי שווה לגודל של כיסוי הקודקודים המינימלי.

הוכחה: באינדוקציה. תמצא את הצמתים עם הדרגה הכי גבוהה, תמצא זיווג לפי משפט הול, תצבע את הזיווג בצבע כלשהו. מורידים קשת אחת ומקבלים דרגה קטנה ב-1 ומפעילים על זה את האינדוקציה.

• Fabrics

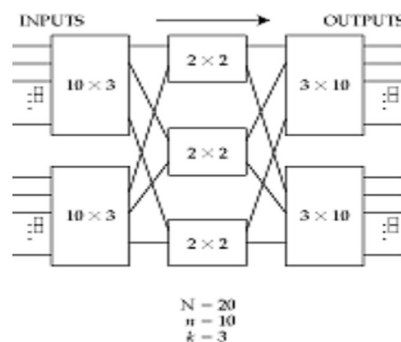
○ Clos רשתות קלוס

Multistage switch – crossbar השתמשנו בנקודת מיתוג אחת לכל הכניסות והיציאות. זה פשוט אבל מצד שני יקר

(עלות ריבועית). לכן במקום לעבור crossbar פעם אחת, נשתמש בהרבה crossbar-ים. כדי למתג שיחה יהיה לנו כמה

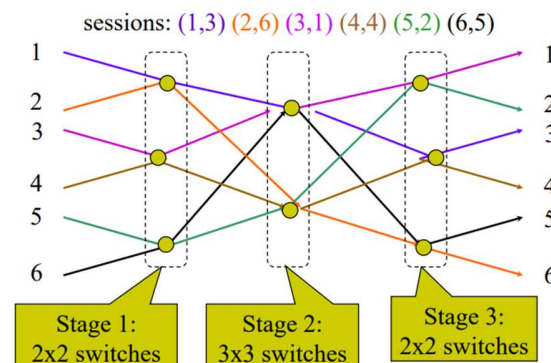
נקודות שבה השיחה תעבור ותמותג. למשל באיור מטה כל הריבועים הם מתגים.

זה חוסך באופן דרמטי בעלות – בשטח של VLSI, במס' נקודות ההצלבה וכו'.



דוגמא: מתג עם 3 רמות של crossbar

כל נקודה צהובה מייצגת תת מתג.



○ Benes

○ Recursive constructions